CSSG EDUCATION TEAM

Pages, routers & in-page nav

From one long page to a site people can move around in

1 Page design | 2 Routers | 3 In-page nav

TODAY

Plan the pages, then build them

Part 1 · On paper

Page design

Which pages exist, how they connect, and what goes on each one.

Part 2 · In React

Routers

Showing a different page for each URL, without reloading.

Part 3 · In HTML + CSS

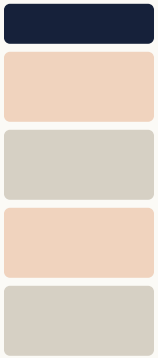
In-page nav

Links that jump to a section of the same page.

Design comes first: you can’t route to pages you haven’t decided on.

PAGE DESIGN · 1

One long page, or many pages?



One page, sections

Everything stacked; the nav scrolls you down.

Good when there's a little content, read top to bottom.

Many pages

Each page has its own URL you can share.

Good when content grows, or one thing deserves its own link.

Most portfolios do both: a one-page home, plus a page per project.

PAGE DESIGN · 2

Turn features into a list of pages

Each page gets one job and one URL.

Page	URL	Its one job
Home	/	Who you are, in 10 seconds
Projects	/projects	Browse everything you've built
Project detail	/projects/:slug	Show one project in depth
Not found	anything else	Say "wrong link" and offer a way home

:slug means "fill in a name here": /projects/weather-app, /projects/pokedex.

PAGE DESIGN · 3

Draw how people move between pages



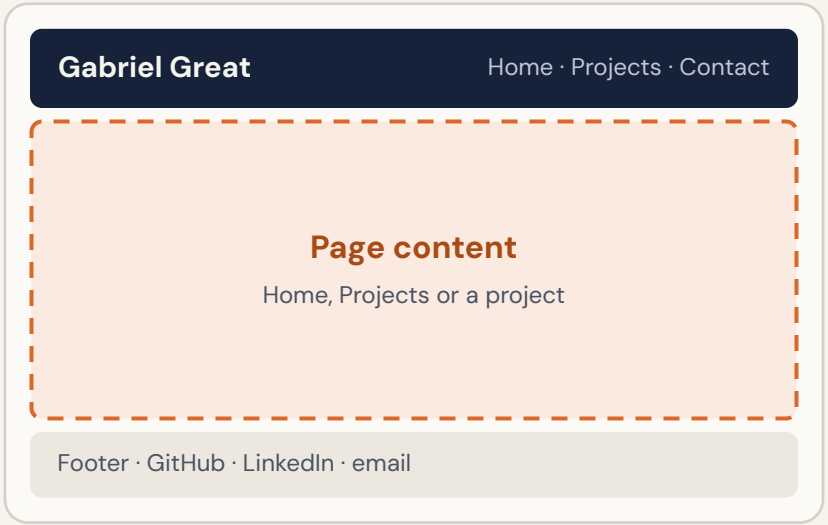
Check your flow

Every page reachable? Every page has a way back?

Any dead ends?

PAGE DESIGN · 4

Some parts stay, some parts change



The layout

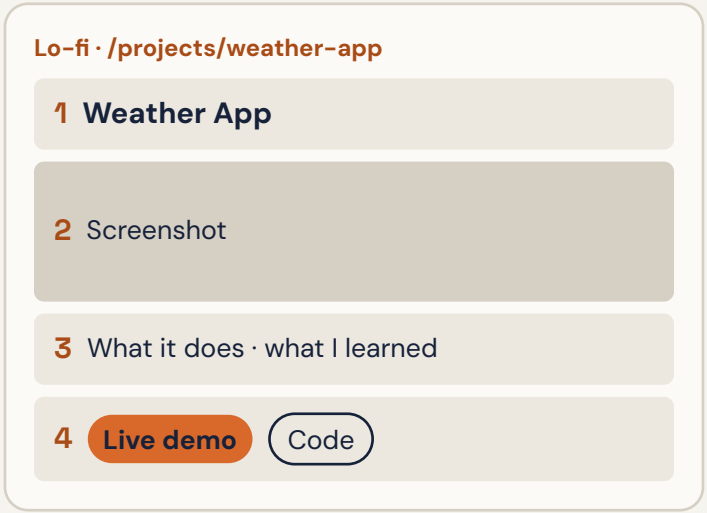
Header, nav and footer. Same on every page, so people never feel lost.

The page

The dashed box. Only this part swaps when the URL changes.

PAGE DESIGN · 5

Order each page by what matters most



Top to bottom = most to least important. Most visitors never scroll far.

One main action per page, styled loudest. Here: Live demo.

Same page type, same order. Every project page follows this template.

PAGE DESIGN · 6

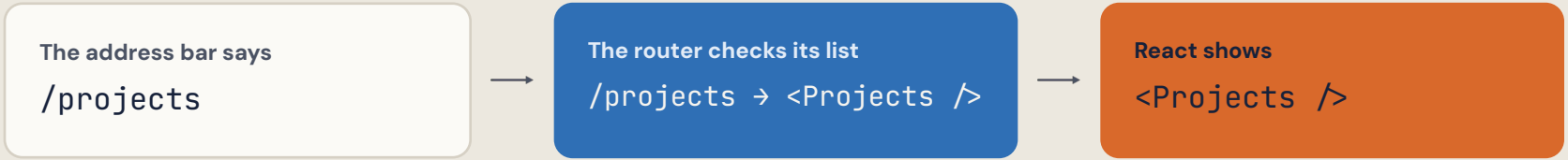
Write down what each page needs to load

Page	Data it needs	Loading	Empty	Error
Home	GitHub profile, 3 featured projects	Gray card shapes	Hide the section	Show the section without GitHub data
Projects	All projects, plus recent repos	Gray card shapes	“Projects coming soon”	“Couldn’t load repos” + Retry
Project detail	One project, found by its slug	Gray title + image	Slug not found: “No such project”	“Couldn’t load” + Back to projects

Same three states from the APIs lesson, decided **per page, before you build.**

ROUTERS · 1

A router picks the page for each URL



Old way: each URL is its own HTML file. Every click loads a whole new page.

React way: one index.html. Clicks swap components; the page never reloads.

ROUTERS · 2

Your page list, written as routes

```
$ npm i react-router

import { BrowserRouter, Routes, Route } from "react-router";

<BrowserRouter>
  <Routes>
    <Route path="/" element={<Home />} />
    <Route path="/projects" element={<Projects />} />
    <Route path="/projects/:slug" element={<Project />} />
    <Route path="*" element={<NotFound />} />
  </Routes>
</BrowserRouter>
```

One Route per page

Same four rows as the page table.

”*” catches the rest

Any URL with no match shows NotFound.

Wrap the app once

BrowserRouter goes in main.tsx.

ROUTERS · 3

Use Link, not <a>, between your own pages

```
<a href="/projects">
```

Full page reload

The browser throws the app away and downloads it again. All state is lost; the screen flashes.

```
<Link to="/projects">
```

Instant swap

The router changes the URL and the component. Nothing reloads; state survives.

Rule: Link for pages inside your site. A plain <a> for other sites, like GitHub or LinkedIn.

Outlet is the box that swaps

```
// Every page inside renders in the layout
<Route element={<Layout />}>
  <Route path="/" element={<Home />} />
  … the other routes
</Route>

function Layout() {
  return (<>
    <nav>
      <NavLink to="/">Home</NavLink>
      <NavLink to="/projects">Projects</NavLink>
    </nav>
    <main><Outlet /></main>
    <footer>…</footer>
  </>);
}
```

<Outlet />

The dashed box from the layout slide. The current page renders here.

NavLink

A Link that knows when it's the current page, so you can style it.

One page component, many projects

```
// URL: /projects/weather-app
function Project() {
  const { slug } = useParams();
  // slug === "weather-app"
  const project = projects.find(
    (p) => p.slug === slug
  );
  if (!project) return <NotFound />;
  return <h1>{project.title}</h1>;
}
```

useParams

Reads the :slug part of the URL.

find by slug

Same idea as finding a post by its ID.

No match?

/projects/banana still matches the route, so you handle it.

ROUTERS · 6 · DEPLOY GOTCHA

Refresh on /projects gives a 404. Why?

Clicking a Link works: the router swaps pages inside the app that's already loaded.

Refreshing asks the host for a file called /projects. There isn't one. Only index.html exists.

The fix: tell the host to send index.html for every URL.

Host	What to add
Vercel	vercel.json: {"rewrites":[{"source":"/(.*)","destination":"/index.html"}]}
Netlify	public/_redirects: /* /index.html 200
GitHub Pages	No rewrite setting. Copy index.html to 404.html when you build.

IN-PAGE NAV · 1

Jump to a section with # and an id

```
<nav>
  <a href="#about">About</a>
  <a href="#projects">Projects</a>
  <a href="#contact">Contact</a>
</nav>

<section id="about">…</section>
<section id="projects">…</section>
<section id="contact">…</section>

/* CSS: glide instead of jump */
html { scroll-behavior: smooth; }
```

#about finds id=”about”

Colors match: the link names the id it scrolls to.

No JavaScript

The browser does it. No reload, no router.

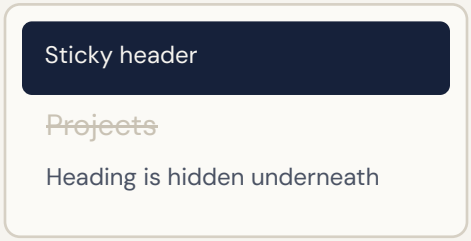
Shareable

yoursite.com/#contact opens right at Contact.

IN-PAGE NAV · 2

A sticky header hides your headings

Problem



Fixed




```
header {
  position: sticky;
  top: 0;
}

section {
  scroll-margin-top: 80px;
}
```

scroll-margin-top tells the browser to stop that far above the section. Match it to your header's height.

IN-PAGE NAV · 3 · OPTIONAL

Highlight the section you're reading

```
const [active, setActive] = useState("");

useEffect(() => {
  const observer = new IntersectionObserver(
    (entries) => entries.forEach((e) => {
      if (e.isIntersecting) setActive(e.target.id);
    }),
    { rootMargin: "-50% 0px -50% 0px" }
  );
  document.querySelectorAll("section[id]")
    .forEach((s) => observer.observe(s));
  return () => observer.disconnect();
}, []);
```

Watch every section

Same Intersection Observer as the fade-in demo.

The middle line

That rootMargin means "whichever section crosses the middle of the screen."

Style the link

Give the nav link whose href matches active an "active" class.

IN-PAGE NAV · 4 · ROUTERS + SECTIONS

“Contact” from another page

You’re on /projects. The Contact section lives on Home. So the link is <Link to="/#contact">

Catch: BrowserRouter opens Home but doesn’t scroll to #contact. You add that yourself.

```
// in Home: scroll to the #hash after the page shows
const { hash } = useLocation();
useEffect(() ⇒ {
  if (hash) document.getElementById(hash.slice(1))
    ?.scrollIntoView();
}, [hash]);
```

ACCESSIBILITY

Navigation everyone can use

Skip link

A first link, “Skip to content,” that jumps to <main id="main">. Keyboard users skip the whole nav.

Say where they are

NavLink adds aria-current="page". Style the current link with more than color, like an underline.

Respect reduced motion

Put smooth scrolling inside @media (prefers-reduced-motion: no-preference).

Links say where they go

“View Weather App,” not “Click here.” Real links, not clickable divs.

HOMEWORK

Plan your pages, then wire them up

1 · On paper

1. Page list: page, URL, one job
2. User flow with every link
3. Layout + one lo-fi per page
4. Data table: loading, empty, error

Commit it to design/README.md

2 · In code

1. Routes for Home, Projects, one project, NotFound
2. A Layout with NavLinks and Outlet
3. Home nav that jumps to 3 sections
4. Deploy, then refresh on /projects

Stretch: highlight the current section